Guía2. Desarrollo Proyecto APT Asignatura Capstone

Campo	Detalle
Estudiante	Andrés Rodrigo Vergara Acevedo
Carrera y sede	Ingeniería en Informática · Duoc UC, sede Viña del Mar
Asignatura	Capstone (PTY4614)
Proyecto APT	DomoVida: Plataforma IoT de Monitoreo y Asistencia para el Adulto Mayor en el Hogar
Etapa del informe	Fase 2 · Sprint 3 (21 de septiembre al 11 de octubre de 2026) · semana 9 de 18 · versión corregida con el avance a la semana 11
Fecha de entrega	7 de octubre de 2026 · versión corregida según retroalimentación: 21 de octubre de 2026
Repositorio	https://github.com/AndresVergara74/domovida-backend
Sistema publicado	Panel: https://domovida-backend.vercel.app · API: https://domovida-backend.onrender.com/docs · Servidor en Chile (Oracle Cloud Santiago): https://domovida.duckdns.org/api/health
Tablero Kanban	https://github.com/users/AndresVergara74/projects/1

Índice

Resumen / Abstract 3
Nota de la versión corregida (21-10-2026) 4
1. Resumen avance Proyecto APT 5
Tabla 1. Evidencias de avance del proyecto DomoVida 7
Figuras 1 a 18. Evidencias del avance 9
2. Monitoreo del Plan de Trabajo 19
3. Ajustes a partir del monitoreo 22
4. Conclusions 25
5. Reflection 26
6. Declaración de uso de inteligencia artificial 26
7. Referencias 26

Resumen / Abstract

Resumen

DomoVida es una plataforma IoT de monitoreo para adultos mayores que viven solos, que detecta caídas y emergencias y alerta al cuidador en tiempo real. Este informe presenta el avance a la semana 9 de 18. El prototipo integra un simulador de 8 sensores, una API FastAPI con persistencia en Supabase/PostgreSQL, un panel React y alertas por WebSocket y notificación push (ntfy). Se aplicaron seudonimización SHA-256, seguridad a nivel de fila (RLS) y una implementación inicial de interoperabilidad HL7 FHIR R4 (recursos Observation, Patient seudonimizado y Bundle, validados con la librería fhir.resources), además de una clave de API para los sensores y el inicio de sesión del cuidador, incorporando medidas técnicas del prototipo alineadas con principios de protección de datos contemplados en la Ley N° 21.719. En 30 caídas simuladas, el 100 % de las alertas llegó al panel por WebSocket, con una latencia promedio de 1.577 ms en Render y de 216 ms en AWS EC2 (entorno experimental de comparación); esta comparación sugiere que la distancia entre servicios es un factor relevante de la latencia observada. El 100 % de las notificaciones llegó al celular del cuidador (30/30) y, sin internet, el modo borde guardó y mostró 10 de 10 caídas. El desarrollo avanzó más rápido que lo planificado, mientras que la documentación ágil se reconstruyó a partir del historial de commits. Versión corregida (21/10/2026): tras la retroalimentación de la Evaluación 2, el backend y su base de datos se desplegaron juntos en Oracle Cloud Santiago (54 ms promedio, 30 de 30 alertas bajo 1 s), la prueba de carga con 20 usuarios concurrentes terminó sin errores y con p95 menor a 200 ms, el modo borde se sincroniza con la nube al volver la conexión (5 de 5) y la Escala SUS aplicada a siete cuidadores reales obtuvo una media de 92,1 (IC 95 %: 83,4 a 100). La documentación ágil se actualiza ahora cada semana.

Abstract

DomoVida is an IoT monitoring platform for older adults who live alone. It detects falls and emergencies and alerts the caregiver in real time. This report presents the progress at week 9 of 18. The prototype integrates an 8-sensor simulator, a FastAPI backend with Supabase/PostgreSQL persistence, a React dashboard, and alerts via WebSocket and push notifications (ntfy). SHA-256 pseudonymization, Row Level Security (RLS), an initial HL7 FHIR R4 interoperability layer (Observation, pseudonymized Patient and Bundle resources, validated with the fhir.resources library), an API key for the sensors and caregiver login were implemented, incorporating technical measures in the prototype aligned with data protection principles contemplated by Chilean Law No. 21,719. Across 30 simulated falls, 100% of the alerts reached the dashboard via WebSocket, with an average latency of 1,577 ms on Render and 216 ms on AWS EC2 (an experimental comparison environment); this comparison suggests that the distance between services is a relevant factor in the observed latency. 100% of the notifications reached the caregiver's phone (30/30) and, without internet, the edge mode stored and displayed 10 out of 10 falls. Development progressed faster than planned, while the agile documentation was rebuilt from the commit history. Corrected version (October 21, 2026): following the Evaluation 2 feedback, the backend and its database were deployed together on Oracle Cloud Santiago (54 ms on average, 30 out of 30 alerts under 1 s), the load test with 20 concurrent users finished with no errors and a p95 below 200 ms, the edge mode now synchronizes with the cloud when the connection returns (5 out of 5), and the SUS evaluation with seven real caregivers reached a mean score of 92.1 (95% CI: 83.4 to 100). The agile documentation is now updated weekly.

Palabras clave: IoT, adulto mayor, detección de caídas, FastAPI, HL7 FHIR, Ley 21.719, Scrum.
Keywords: IoT, older adults, fall detection, FastAPI, HL7 FHIR, Chilean Law 21,719, Scrum.

Nota de la versión corregida (21 de octubre de 2026)

Esta versión incorpora la retroalimentación de la Evaluación 2. El texto original (avance a la semana 9) se conserva; los puntos observados por el docente se actualizaron en cada sección, y la tabla siguiente resume cada corrección con su evidencia en el repositorio.

Tabla 0. Correcciones realizadas según la retroalimentación de la Evaluación 2

N°	Observación del docente	Corrección realizada y resultado	Evidencia
1	Desplegar backend y base de datos en la misma región para alcanzar la meta de latencia	Backend y base de datos en la misma máquina en Oracle Cloud Santiago, con HTTPS. Latencia promedio 54 ms; 30 de 30 alertas bajo 1 s (11/10). Meta < 1 s cumplida.	PR-01c (Figura 18); registro de pruebas, ajustes 22 y 23
2	Ejecutar la prueba de carga	PR-04 con Locust (20/10): 20 usuarios concurrentes durante 2 minutos, 976 solicitudes y 0 errores. p95 de 57 ms en /api/sensor-data y de 110 ms en /api/eventos (meta < 200 ms). Observación: el p99 llegó a 1,2–2,3 s (ajuste 24).	PR-04_reporte_locust.html y PR-04_stats.csv; commit 455a5b0
3	Actualizar el informe con la sincronización del modo borde ya implementada	Implementada el 08/10 (HU-03): almacenamiento y reenvío con UUID. Sin internet las caídas quedan pendientes en SQLite y al volver la conexión se sincronizan sin duplicados (5 de 5).	PU-07 (13/13) y PR-03b (5/5); commits 0a3f3df y 1178ebf
4	Depurar el repositorio (archivo init.pyNew-Item y estructura duplicada backend/app)	Eliminados el esqueleto sin uso backend/app y el archivo residual. Render y Docker ejecutan backend/main.py, por lo que el sistema no cambió.	Commit 1a82453; ajuste 25
5	Sostener la documentación ágil semanal (tablero, fichas de sprint y registro de pruebas)	Cierre del Sprint 3, ficha del Sprint 4 y bitácora semanal (Tabla H); Product Backlog al 20/10 con cuatro historias nuevas (HU-20 a HU-23) surgidas de las sesiones SUS; tablero Kanban actualizado. Desde el 25/10 la bitácora se actualiza cada domingo.	Fichas_Sprint_1-3_DomoVida.docx, product_backlog.xlsx; commit e0c2a07
6	Aplicar la evaluación SUS con cuidadores reales bajo consentimiento informado, con estadísticos descriptivos e intervalos de confianza	Siete cuidadores reales (15 al 21/10), con consentimiento en el formulario y sin datos personales (códigos P01 a P07). Resultados en el párrafo siguiente.	Notas de sesión P01 a P07 (usabilidad/sesiones); formulario SUS

Resultados SUS (n = 7). Puntajes: 80; 77,5; 100; 100; 95; 95 y 97,5. Media 92,1; desviación estándar 9,4; mediana 95; mínimo 77,5 y máximo 100. Intervalo de confianza del 95 % para la media, con distribución t de Student (6 grados de libertad): 83,4 a 100,8, acotado a 83,4 a 100 por el máximo de la escala. Todo el intervalo supera la meta del proyecto (> 70) y el umbral de 68 de la escala, y la media se ubica sobre 80,3, nivel considerado excelente (Sauro, 2011; Bangor et al., 2009). Tasa de éxito por tarea: T1 6/7 (P01 no pudo ingresar desde el celular por la contraseña; se corrigió antes de la siguiente sesión), T2 a T4 7/7 y T5 6/6 registradas. Los hallazgos de las sesiones se incorporaron al Product Backlog como las historias HU-20 a HU-23.

1. Resumen avance Proyecto APT

A continuación, encontrarás distintos campos que deberás completar con la información solicitada.

Resumen de avance proyecto APT	A la semana 9 de 18 (Sprint 3, del 21 de septiembre al 11 de octubre de 2026), DomoVida cuenta con un prototipo funcional desplegado en la nube que cubre el flujo crítico completo: caída → detección → alerta → atención del cuidador. Los tres objetivos específicos presentan avance: **OE1 (captura y detección en el borde): alto grado de implementación y validación dentro del alcance actual del prototipo.** El simulador de 8 sensores (acelerómetro, movimiento, gas, humo, dos sensores de apertura, sensor cardíaco y botón de pánico) envía lecturas cada 5 segundos; las reglas de umbral marcan los eventos críticos y SQLite3 respalda los datos localmente. En la prueba de continuidad sin conexión (PR-03, 03/10), con internet cortado, el modo borde guardó en SQLite y mostró en el panel local 10 de 10 caídas. La regla de la puerta principal se trasladó al backend: alerta solo si se abre de noche o si queda abierta 10 minutos o más (PU-04, 10/10). La sincronización automática con la nube al volver la conexión se implementó el 08/10 (HU-03): 5 de 5 caídas sincronizadas sin duplicados (PR-03b). **OE2 (backend, datos y protección de datos): alto grado de implementación y validación dentro del alcance actual del prototipo.** La API FastAPI (Ramírez, 2023) está desplegada en Render y sincroniza los eventos con Supabase/PostgreSQL (Supabase Inc., 2024). Cada evento crítico genera su alerta con la severidad correcta (prueba PI-02) y las políticas RLS impiden el acceso anónimo (PI-03). El RUT se seudonimiza con SHA-256 (NIST, 2015) con sal (PU-03) y se realizó una implementación inicial de HL7 FHIR R4 que representa los eventos como recursos Observation agrupados en Bundle, validados con la librería fhir.resources (PI-04). Además, el sistema se empaquetó en Docker (Docker Inc., 2024) y se desplegó una réplica en AWS EC2, utilizada exclusivamente como entorno experimental de comparación; la arquitectura de producción del prototipo es Vercel + Render + Supabase + ntfy. Entre el 03 y el 05/10 se completaron los pendientes: el recurso FHIR Patient seudonimizado, correctamente estructurado y validado mediante la librería fhir.resources, sin nombre, RUT ni dirección (PI-05, 8/8); una clave de API que exigen los sensores (PS-01, 3/3 en producción); el inicio de sesión del cuidador con Supabase Auth para atender alertas, que registra quién atendió cada una (PS-01b, 3/3 en producción), y RLS en la tabla de sensores (PI-03b). El 06/10 se reforzó la calidad de los datos: los cambios de la base pasaron a migraciones versionadas en el repositorio, todas las fechas se guardan en UTC con zona horaria y la tabla alertas quedó como fuente única de la atención, lo que corrigió 1.463 registros inconsistentes (PU-05, PU-06, PI-06 y PI-07). **OE3 (panel del cuidador y alertas): en curso, con avance mayor al planificado.** El panel React (Meta Platforms, 2024) en Vercel muestra 4 pestañas, el mapa Leaflet y el registro de atención de cada alerta. En 30 caídas simuladas, el 100 % de las alertas llegó al panel por WebSocket (Fette y Melnikov, 2011; PR-01); en la prueba PR-02, los 30 avisos ntfy (Heckel, 2024) llegaron al celular del cuidador (100 %, meta > 95 %). El panel ahora distingue los componentes simulados, muestra la base de datos en uso y la hora de atención de cada alerta. La evaluación de usabilidad con la Escala SUS se adelantó al Sprint 4: siete cuidadores reales entre el 15 y el 21/10, con media 92,1 e IC 95 % de 83,4 a 100 (meta > 70; ver Tabla 0). El desarrollo avanzó más rápido que lo planificado en la Guía 1.5, que proyectaba solo el OE1 a esta fecha. En cambio, la documentación ágil se atrasó y se reconstruyó a partir del historial de commits.

Objetivos	Se mantienen el objetivo general y los tres objetivos específicos definidos en la Guía 1.5. Los ajustes son de alcance y de indicadores, no de propósito: • **OE2, interoperabilidad HL7 FHIR (adelantada):** la Guía 1.5 planificaba los endpoints FHIR para la etapa post-prototipo. Se adelantó una implementación inicial al Sprint 3: el backend expone los eventos como recursos Observation y Bundle FHIR R4, validados con la librería fhir.resources en la prueba PI-04, y el 05/10 se agregó el recurso Patient seudonimizado (PI-05). • **OE3, latencia de la alerta (indicador precisado):** se mantiene la meta de menos de 1 s, pero se precisa su condición de medición. Con backend y base de datos en regiones distintas (Render en EE. UU. y Supabase en Brasil), la alerta tarda en promedio 1.577 ms; con ambos en la misma máquina (AWS EC2), tarda 216 ms y cumple la meta en el 100 % de los casos de la prueba realizada (30/30). La meta se evaluó el 11/10 en Oracle Cloud Santiago, con backend y base de datos en la misma máquina: 54 ms promedio y 30 de 30 alertas bajo 1 s (PR-01c). • **Alcance:** la detección con Machine Learning (Isolation Forest) y el botón de pánico portátil se mantienen como trabajo futuro. Para la pulsera se definió una ruta en tres etapas: hardware abierto existente, luego una pulsera propia con ESP32.
Metodología	Se mantiene Scrum (Schwaber y Sutherland, 2020) adaptado a un desarrollador, con seis sprints de tres semanas. Los ajustes son: • **Registro del trabajo:** la bitácora en Obsidian no pudo usarse con la cuenta institucional. Se reemplazó por artefactos dentro del repositorio GitHub: fichas de sprint y retrospectivas, Definition of Done y registro de pruebas. Los Sprints 1 a 3 se reconstruyeron a partir del historial de commits; desde el 20/10 el avance se registra cada semana en la bitácora del sprint en curso. • **Tablero Kanban:** el Product Backlog se gestiona en GitHub Projects, ahora con acceso público para su revisión. • **Pruebas reproducibles:** cada prueba se ejecuta con un script versionado (por ejemplo, PR-01_latencia.js y PI-04_validacion_fhir.py) y queda registrada con fecha, resultado y evidencia. • **Entornos de prueba comparativos:** se incorporó un entorno de prueba con Docker en AWS EC2 para comparar el comportamiento del sistema frente a Render + Supabase. Se intentó un tercer entorno en Oracle Cloud (región Santiago), pero la región no tenía capacidad para las máquinas gratuitas en dos intentos (03 y 05/10), por lo que se posterga. AWS EC2 se utilizó exclusivamente como entorno experimental de comparación y no corresponde a la arquitectura de producción del prototipo (Vercel + Render + Supabase + ntfy); Actualización: el 11/10 se desplegó en Oracle Cloud Santiago con la forma gratuita E2.1.Micro y HTTPS, donde se ejecutaron PR-01c y PR-04. • **Verificación de despliegues:** después de cada git push se revisa que Vercel y Render queden publicados, como resultado de la retrospectiva del Sprint 3. • **Pruebas de seguridad en producción:** las pruebas PS-01 y PS-01b se ejecutan contra el sistema publicado, y la clave se verifica comparando una huella SHA-256 parcial, sin exponerla.
Evidencias de avance	Se presentan veintiuna evidencias (Tabla 1; E17 a E21 agregadas en la versión corregida), seleccionadas porque demuestran con datos medibles el avance de cada objetivo específico y del proceso ágil. Todas están disponibles en el repositorio GitHub (carpeta docs/fase2/evidencias_proyecto) y en el sistema publicado; las principales se adjuntan a continuación como Figuras 1 a 18. **Resguardo de la calidad.** La calidad del proyecto se resguardó con prácticas propias de la ingeniería de software:

	• Definition of Done de 7 criterios, usada como referencia para dar por terminada cada historia de usuario. • Control de versiones en GitHub, con commits descriptivos que, desde el Sprint 3, usan en su mayoría los prefijos feat, fix, docs y perf. • Pruebas planificadas (unitarias, de integración y de rendimiento) con metas medibles, ejecutadas con scripts reproducibles y registradas con fecha y resultado. La prueba PR-01 se repitió después del ajuste para comprobar su efecto. • Privacidad por diseño: seudonimización, RLS y notificaciones sin datos sensibles. • Estándares abiertos: HL7 FHIR R4 (HL7 International, 2019), SNOMED CT (SNOMED International, 2024) y LOINC (Regenstrief Institute, 2024). • Registro honesto de los hallazgos: se documentaron también los resultados que no cumplen la meta (latencia en Render) y las brechas de seguridad detectadas (API sin autenticación, ya corregida; tabla sin RLS, ya corregida; falta de https en AWS), cada una con su plan de acción.

Tabla 1. Evidencias de avance del proyecto DomoVida

N°	Evidencia	Qué demuestra	OE
E1	Panel en Vercel con datos del simulador (Figura 1)	8/8 sensores enviando datos en tiempo real a la nube	OE1
E2	Alerta en tiempo real y atención del cuidador (Figuras 2 y 3)	Flujo crítico completo: caída → alerta → atención registrada	OE3
E3	Notificaciones ntfy en el celular (Figura 4)	El aviso llega al celular del cuidador con hora de Chile (recepción verificada visualmente)	OE3
E4	Prueba PR-01 en tres mediciones (Figuras 5, 6 y 7)	30/30 alertas por WebSocket; latencia de 1.903 → 1.577 ms tras el ajuste y 216 ms en AWS EC2	OE3
E5	Trigger de alertas en Supabase, PI-02 (Figura 8)	Cada evento crítico genera su alerta automáticamente, con la severidad correcta	OE2
E6	RLS con rol anónimo, PI-03 (Figura 9)	Un usuario no autorizado no puede leer datos (0 filas)	OE2
E7	Seudonimización SHA-256, PU-03 (Figura 10)	El consentimiento no guarda el RUT en texto plano (principio de seudonimización, Ley N° 21.719; República de Chile, 2024)	OE2
E8	Bundle HL7 FHIR R4 en producción, PI-04 (Figura 11)	Implementación inicial de interoperabilidad: eventos como recursos Observation agrupados en Bundle FHIR R4, validados con la librería fhir.resources	OE2
E9	Tablero Kanban en GitHub Projects (Figura 12)	Gestión ágil activa: 8 elementos terminados, 1 en curso y 4 pendientes	Metodología
E10	Fichas de sprint y retrospectivas (docs/fase2/evidencias_proyecto/documentacion/agil)	Ciclo Scrum completo: planificar, ejecutar, revisar y mejorar	Metodología
E11	Prueba PR-02 de entrega ntfy (Figura 13)	30 de 30 avisos de caída recibidos en el celular del cuidador (100 %)	OE3
E12	Prueba PR-03 sin conexión (Figura 14)	Sin internet, el modo borde guarda y muestra 10 de 10 caídas	OE1
E13	Pruebas PS-01 y PS-01b en producción (Figura 15)	Solo los sensores con clave envían datos y solo el cuidador con sesión atiende alertas (3/3 y 3/3)	OE2
E14	Pruebas PI-05 y PI-03b (registro de	Recurso FHIR Patient correctamente	OE2

Nº	Evidencia	Qué demuestra	OE
	pruebas)	estructurado y validado mediante la librería fhir.resources (8/8), sin datos identificatorios directos; tabla de sensores con RLS (0 filas para el rol anónimo)	
E15	Prueba PU-04 y estado del sistema en el panel (Figura 16)	Regla de la puerta principal (10/10) y componentes simulados identificados en el panel	OE1 y OE3
E16	Migraciones de la base de datos, PI-06 y PI-07 (Figura 17)	Fechas en UTC sin pérdida de datos y 1.463 inconsistencias de atención corregidas a 0, con respaldo y medición antes y después	OE2
E17	Prueba PR-01c en Oracle Cloud Santiago (Figura 18)	Con backend y base de datos en la misma máquina en Chile, la alerta llega en 54 ms promedio y 30 de 30 bajo 1 s	OE3
E18	Prueba PR-03b y PU-07 (registro de pruebas)	El modo borde sincroniza con la nube al volver internet: 5 de 5 caídas, sin duplicados y con su hora original	OE1 y OE2
E19	Prueba de carga PR-04 (reporte de Locust)	20 usuarios concurrentes, 976 solicitudes, 0 errores y p95 bajo 200 ms en los endpoints REST	OE2
E20	Sesiones SUS P01 a P07 (usabilidad/sesiones)	Usabilidad validada con siete cuidadores reales: media 92,1 (IC 95 %: 83,4 a 100) y hallazgos que alimentaron el backlog (HU-20 a HU-23)	OE3
E21	Fichas del Sprint 4, bitácora semanal y backlog al 20/10	Documentación ágil sostenida semana a semana, con trazabilidad entre hallazgos, historias y commits	Metodología

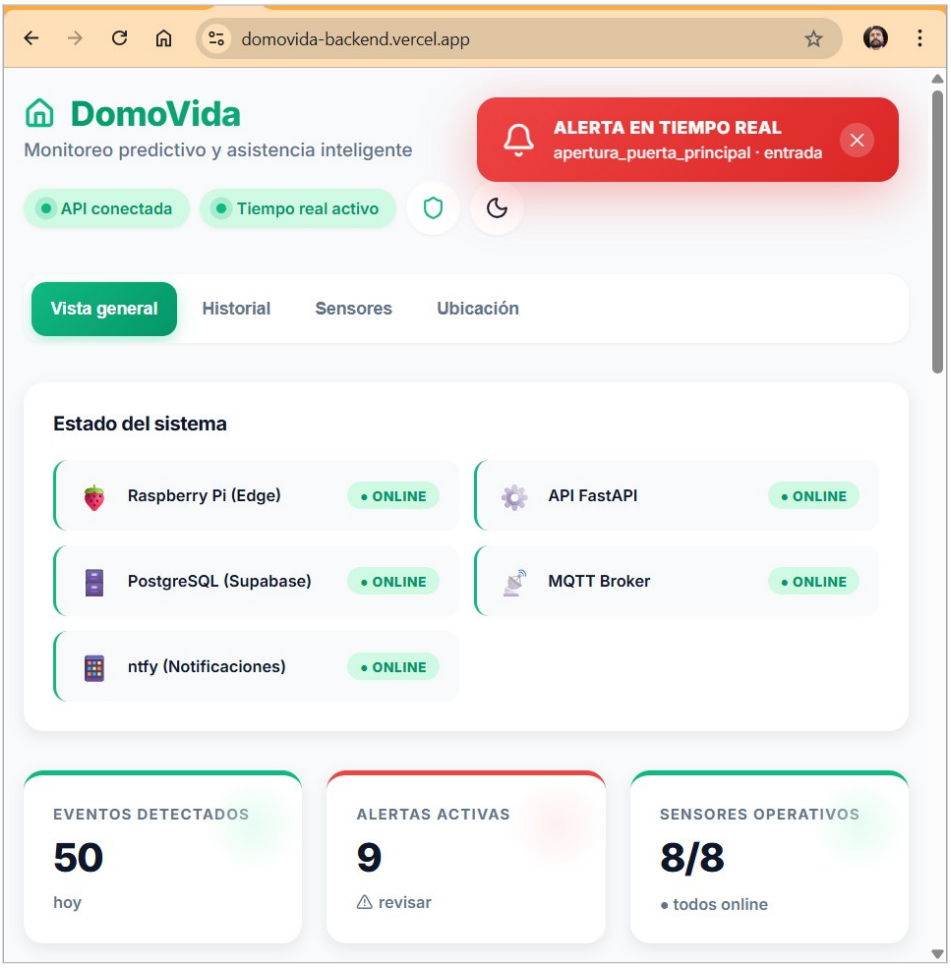


Figura 1. Vista general del panel con los 8 sensores en línea y eventos del día (E1).

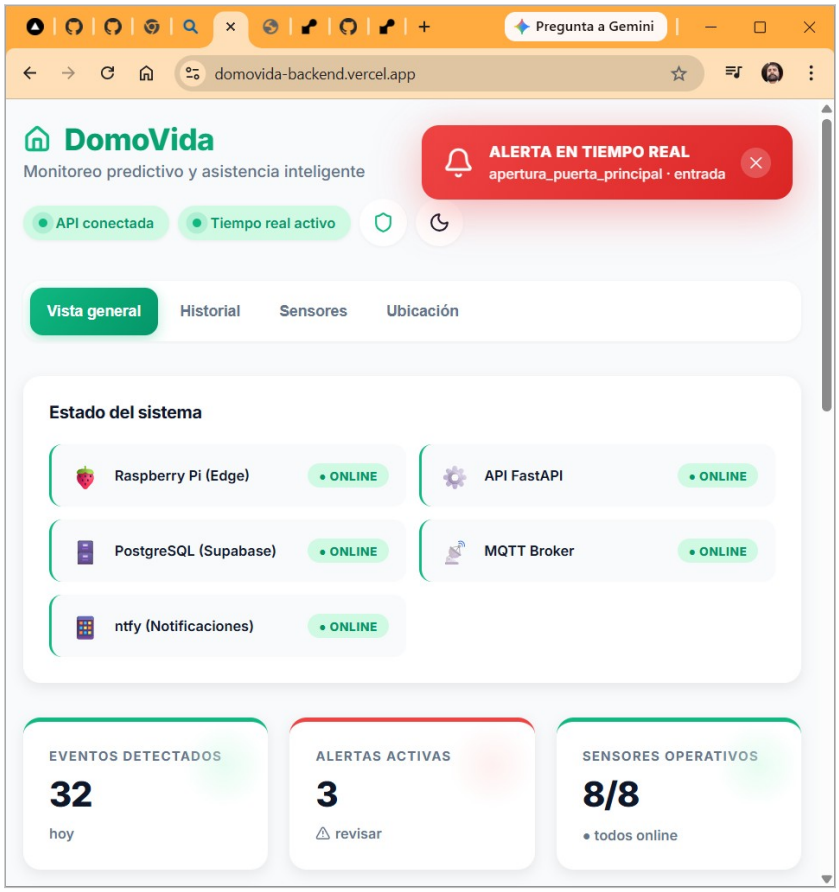


Figura 2. Alerta en tiempo real recibida por WebSocket en el panel del cuidador (E2).

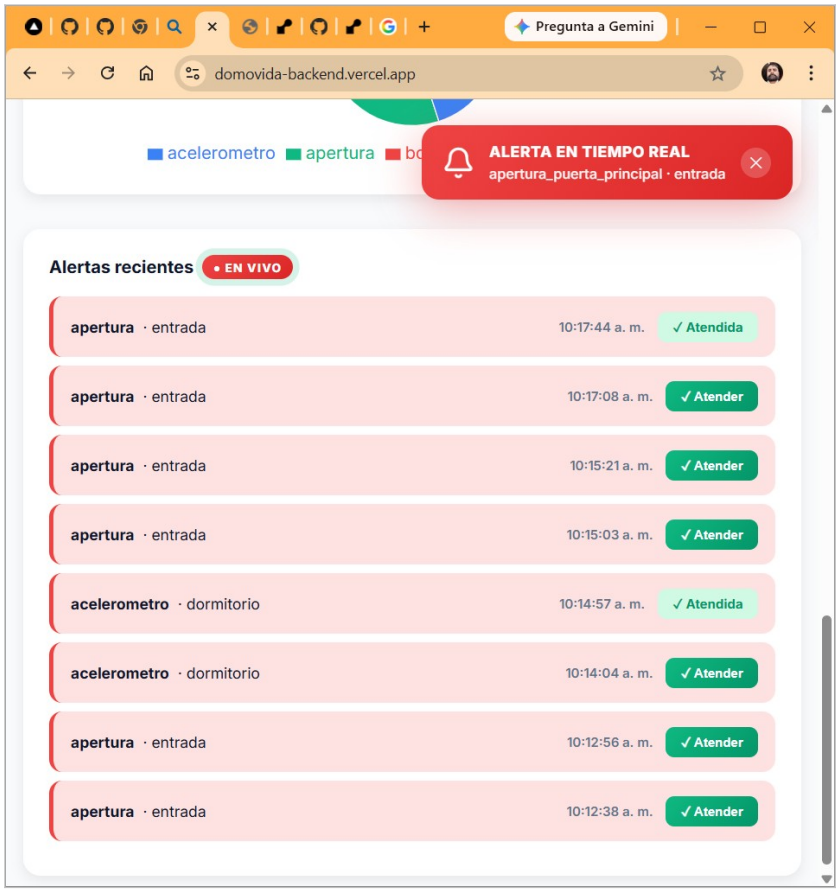


Figura 3. Alertas atendidas por el cuidador, con registro de su estado (E2).

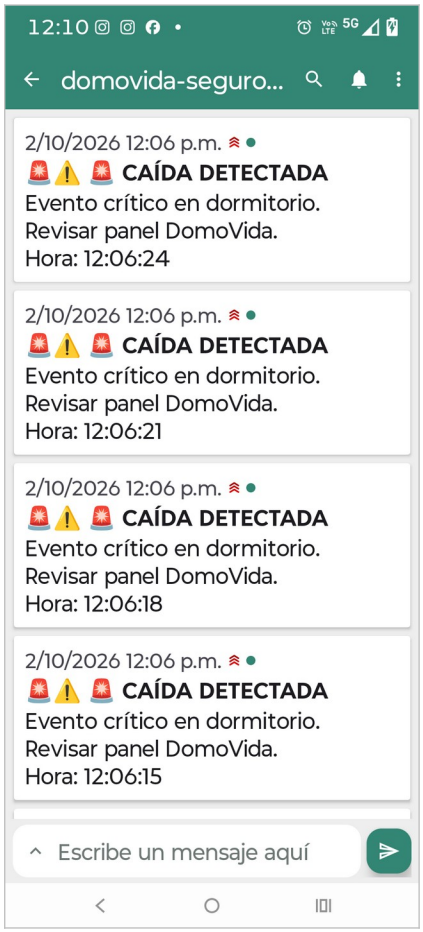


Figura 4. Notificaciones ntfy de caída recibidas en el celular del cuidador (E3).

(index)	Value
Caídas enviadas	30
Alertas recibidas por WebSocket	'30 (100%)'
Promedio (ms)	1903
Mínimo (ms)	1729
Máximo (ms)	2985
Percentil 95 (ms)	2198
Bajo 1 s	'0 de 30'
Bajo 3 s	'30 de 30'

Figura 5. PR-01 en Render + Supabase antes del ajuste: promedio 1.903 ms (E4).

(index)	Value
Caídas enviadas	30
Alertas recibidas por WebSocket	'30 (100%)'
Promedio (ms)	1577
Mínimo (ms)	1445
Máximo (ms)	2410
Percentil 95 (ms)	1879
Bajo 1 s	'0 de 30'
Bajo 3 s	'30 de 30'

Figura 6. PR-01 en Render + Supabase después del ajuste: promedio 1.577 ms (E4).

(index)	Value
Entorno	'AWS EC2 (Docker + SQLite)'
Caídas enviadas	30
Alertas recibidas por WebSocket	'30 (100%)'
Promedio (ms)	216
Mínimo (ms)	155
Máximo (ms)	419
Percentil 95 (ms)	300
Bajo 1 s	'30 de 30'
Bajo 3 s	'30 de 30'

Figura 7. PR-01 en AWS EC2 (Docker + SQLite): promedio 216 ms, 30 de 30 bajo 1 s (E4).

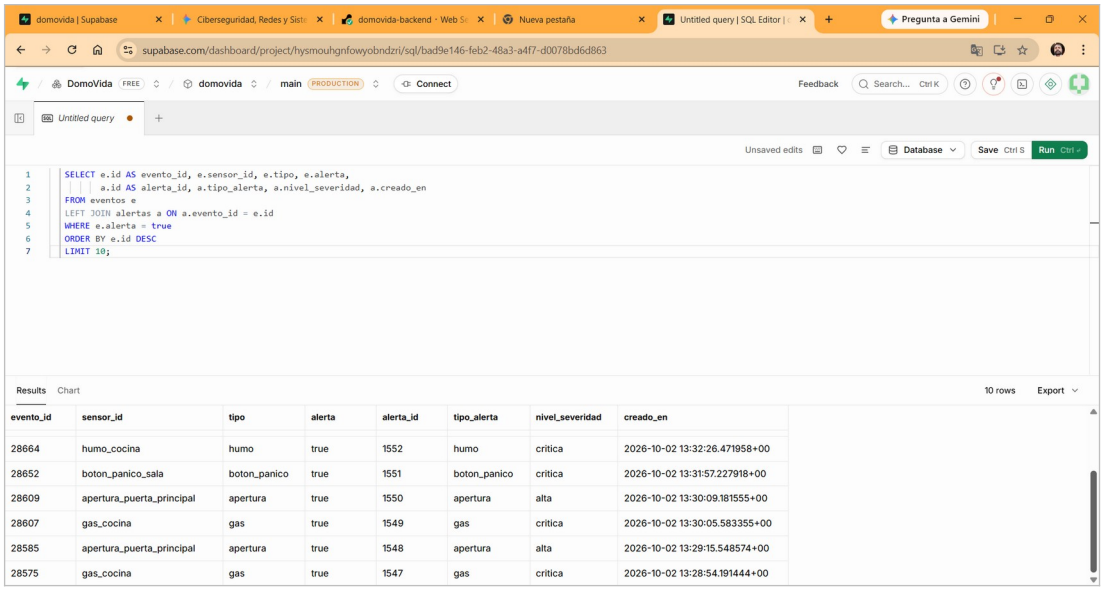


Figura 8. PI-02: cada evento con alerta tiene su registro en la tabla alertas (E5).

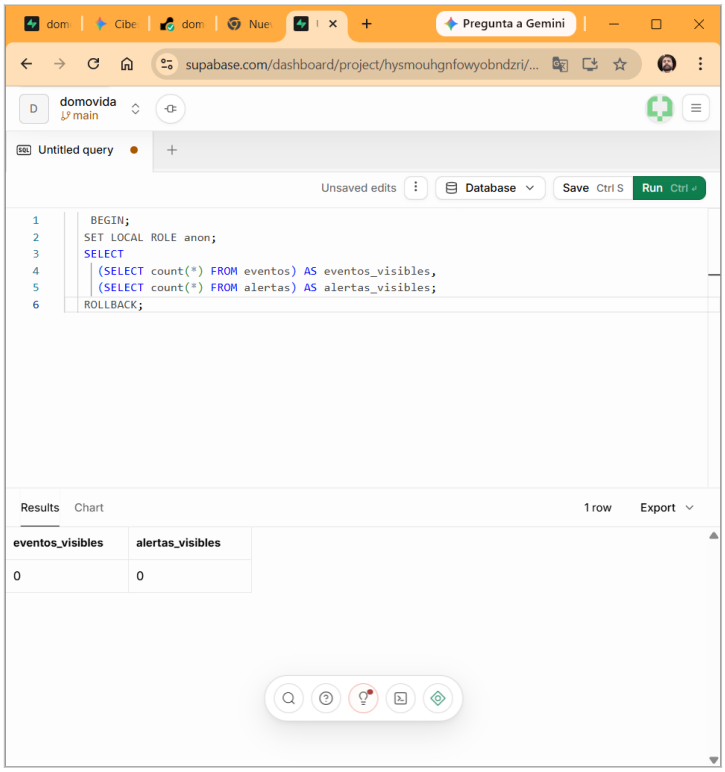


Figura 9. PI-03: con el rol anónimo no se ve ninguna fila de eventos ni de alertas (E6).



Figura 10. PU-03: el consentimiento guarda solo iniciales y el hash SHA-256 del RUT (E7).

```
{ "resourceType": "Bundle", "type": "searchset", "total": 3,
  "entry": [ { "resource": { "resourceType": "Observation", "status":
"final",
      "code": { "coding": [ { "system": "http://snomed.info/sct",
                              "code": "1912002", "display": "Fall
(event)" } ] },
      "subject": { "reference": "Patient/domovida-p001",
                   "display": "Adulto mayor (seudonimizado)" },
      "interpretation": [ { "coding": [ { "code": "AA" } ] } ],
      "effectiveDateTime": "2026-10-02T15:06:23+00:00" } } ] }
```

Figura 11. Extracto del Bundle FHIR R4 servido en producción por GET /api/fhir/Observation (E8).

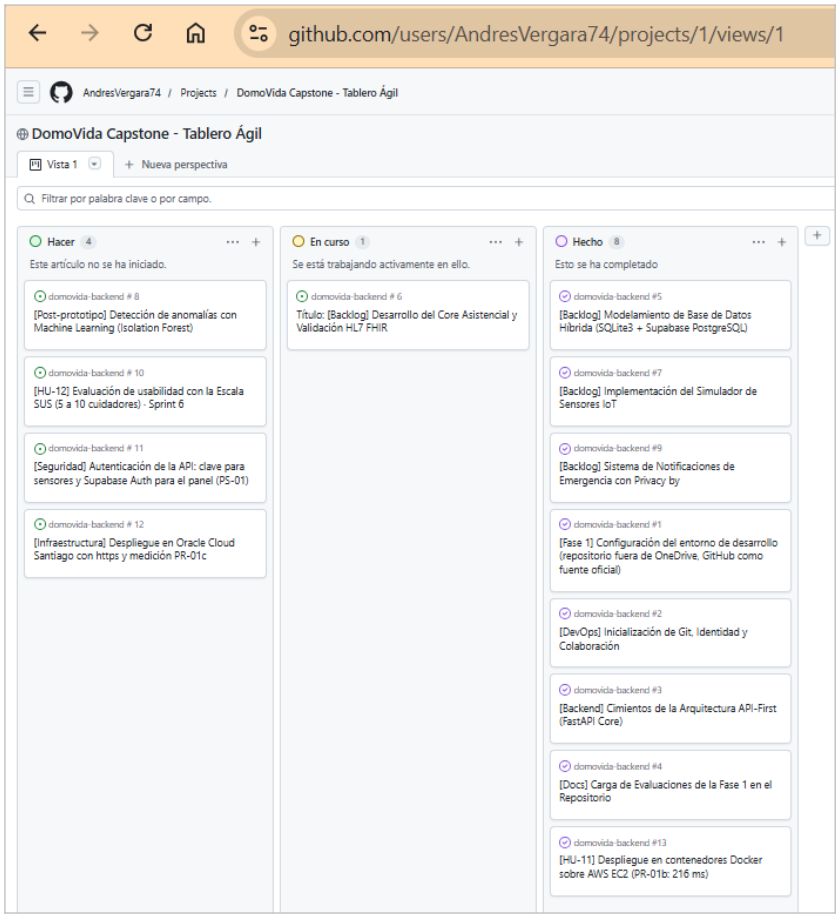


Figura 12. Tablero Kanban del proyecto al 2 de octubre de 2026 (E9).

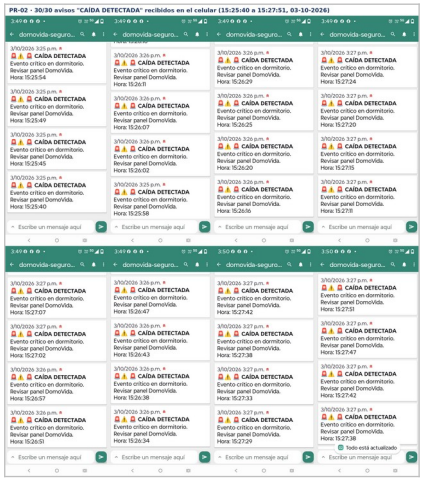


Figura 13. PR-02: los 30 avisos «CAÍDA DETECTADA» recibidos en el celular del cuidador, 03-10-2026 (E11).

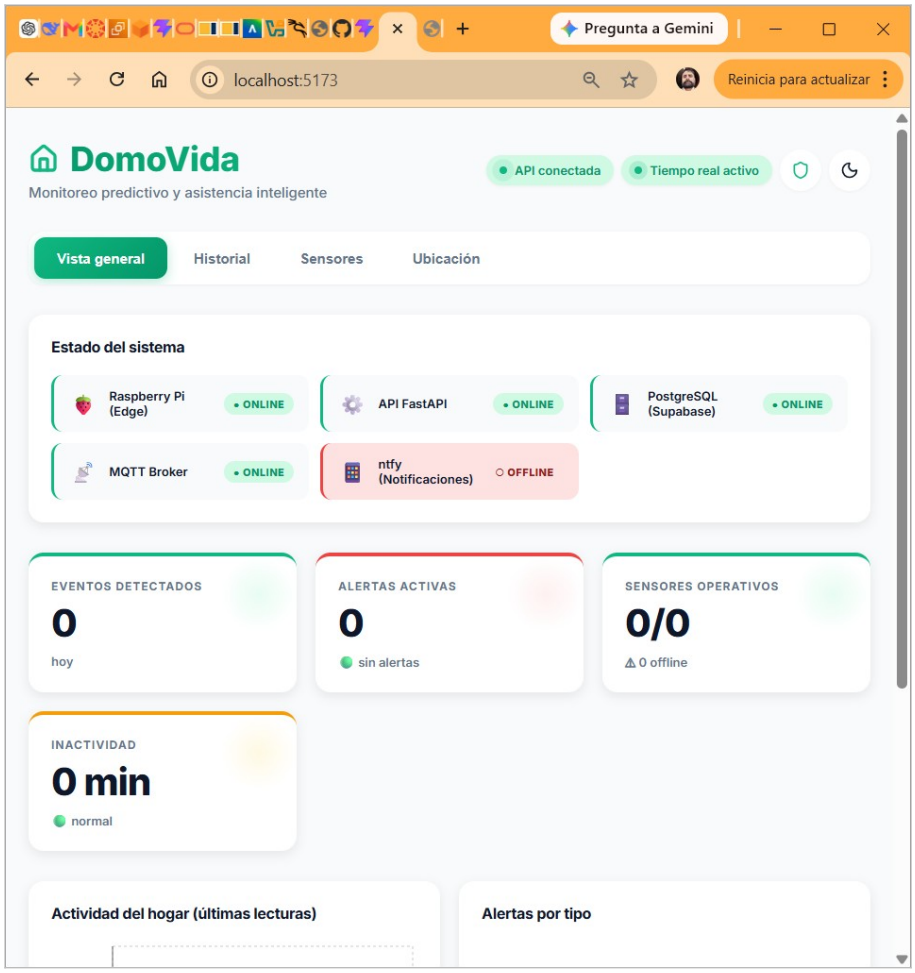


Figura 14. PR-03: panel local sin internet; ntfy fuera de línea y el resto del sistema operativo (E12).

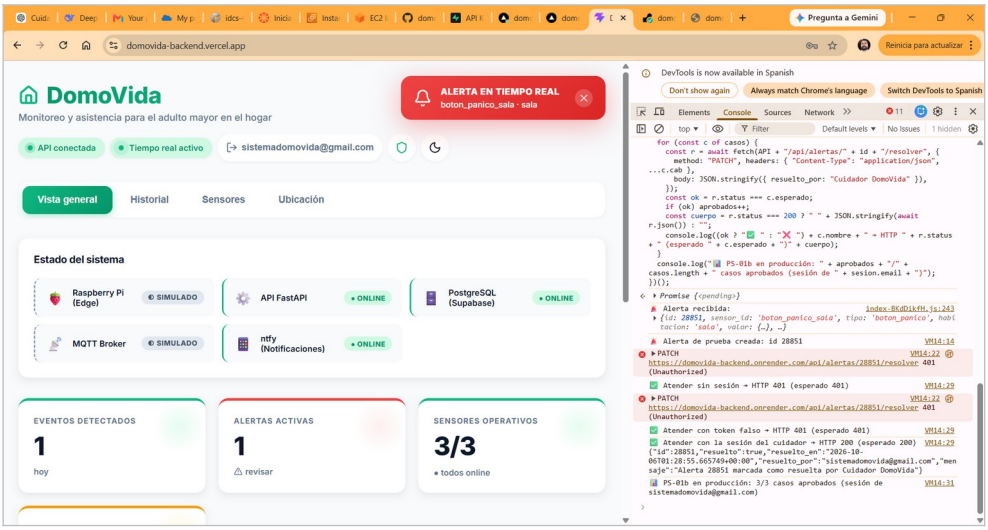


Figura 15. PS-01b en producción: sin sesión y con token falso 401; con la sesión del cuidador 200 y su correo registrado (E13).

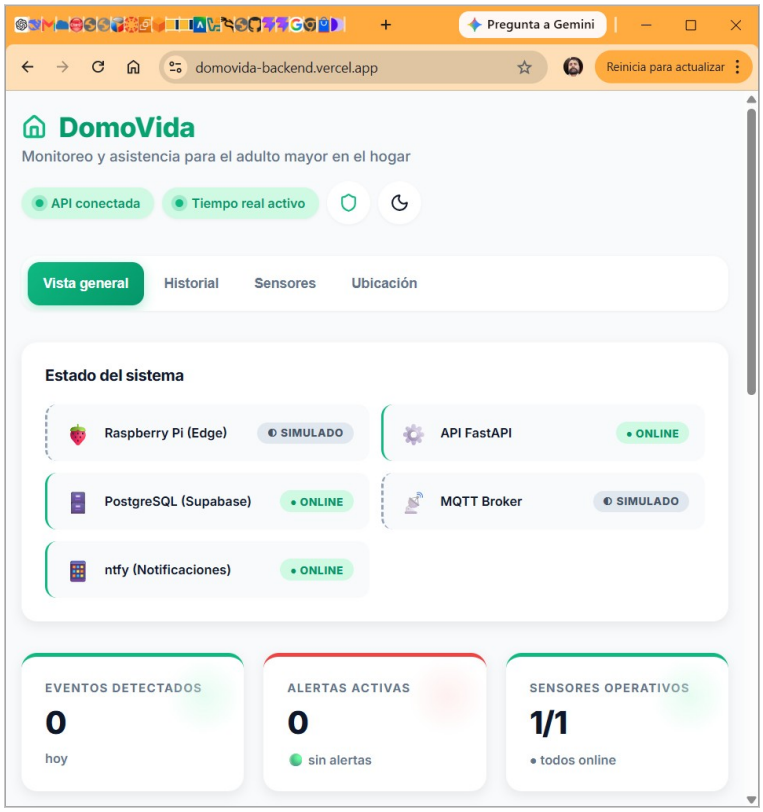


Figura 16. Estado del sistema en el panel: Raspberry Pi y MQTT marcados como simulados (E15).

Antes de la migración 002 (revisión previa)

eventos_con_alertas_duplicadas	eventos_sin_alerta	diferencias_atencion	alertas_sin_evento
0	0	1463	0

Después de la migración 002 (verificación final)

eventos_sin_alerta	diferencias_atencion	trigger_antiguo	migraciones
0	0	0	000, 001, 002

Figura 17. PI-07: consulta en Supabase antes y después de la migración 002; las diferencias de atención entre eventos y alertas bajan de 1.463 a 0 (E16).

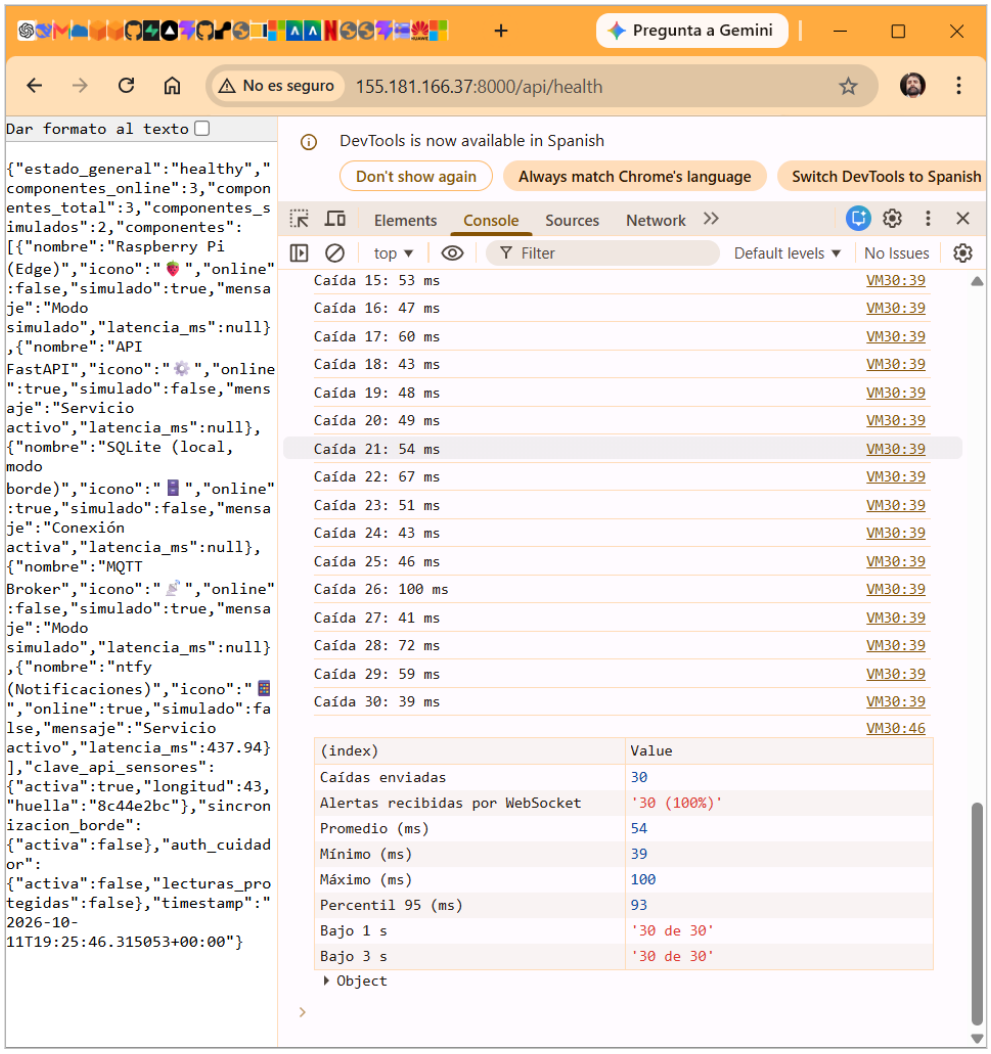


Figura 18. PR-01c en Oracle Cloud Santiago (backend + SQLite en la misma máquina): promedio 54 ms, 30 de 30 bajo 1 s (E17).

2. Monitoreo del Plan de Trabajo

Examina cuidadosamente tu plan de trabajo, enfocándote especialmente en la columna de estado de avance y ajustes.

Plan de Trabajo							
Competencia o unidades de competencias	Actividades	Recursos	Duración de la actividad	Responsable[1]	Observaciones	Estado de avance	Ajustes
Gestionar proyectos informáticos	**Sprint 1: Levantamiento y planificación.** Análisis de la problemática, requisitos, historias de usuario, Product Backlog, repositorio y bitácora.	GitHub Projects, Git, Obsidian, Conda	3 semanas (S1–S3)	Andrés Vergara	Facilitador: guías de la asignatura y experiencia previa. Obstáculo: requisitos amplios e incorporación tardía (19/08).	**Completado**	Repositorio creado el 21/08. La bitácora en Obsidian se reemplazó por artefactos en GitHub (ver actividad 7).
Desarrollar soluciones de software; Construir modelos de datos	**Sprint 2: Diseño de arquitectura y datos.** Arquitectura Edge-Cloud, flujo crítico, modelo entidad-relación, riesgos, Guía 1.5 y presentación.	draw.io, VS Code, PowerPoint	3 semanas (S4–S6)	Andrés Vergara	Facilitador: retroalimentación del profesor guía. Obstáculo: cambios de enfoque.	**Completado**	En este sprint se adelantó la implementación del OE1. La Guía 1.5 se actualizó a la v7 (02/10), incorporando Docker y una infraestructura AWS para pruebas comparativas.
Desarrollar soluciones de software	**Sprint 3: Captura y detección (OE1).** Simulador de 8 sensores, reglas de detección, persistencia SQLite3 y API base.	Python, FastAPI, SQLite3	3 semanas (S7–S9)	Andrés Vergara	Facilitador: documentación de FastAPI (Ramírez, 2023) y SQLite (Hipp, 2024). Obstáculo: sin hardware físico (sensores simulados).	**Completado** (adelantado)	Implementación terminada durante el Sprint 2. Simulador configurable por variable de entorno para enviar datos a Render o AWS (commit 1c8c203). Prueba de continuidad sin conexión ejecutada el 03/10 (PR-03, 10/10). Regla de la puerta principal trasladada al backend (PU-04, 05/10).

[1] En caso de que el Proyecto APT sea grupal, en esta columna deben indicar el nombre de los responsables de cada tarea o actividad. Esto posteriormente permitirá diferenciar la evaluación por cada integrante.

Plan de Trabajo							
Competencia o unidades de competencias	Actividades	Recursos	Duración de la actividad	Responsable	Observaciones	Estado de avance	Ajustes
Construir modelos de datos; Desarrollar soluciones de software	**Sprint 4: Nube, interoperabilidad y panel (OE2 y OE3).** Supabase, HL7 FHIR, seudonimización, RLS, panel React, WebSocket, ntfy y despliegue.	Supabase, Render, Vercel, React, ntfy, Docker, AWS Academy	3 semanas (S10–S12)	Andrés Vergara	Facilitador: planes gratuitos y documentación abierta. Obstáculo: restricciones de red (puerto 5432 bloqueado).	**En curso** (adelantado)	Ejecutado en gran parte en el Sprint 3. Neon → Supabase (Session Pooler IPv4, puerto 6543). HL7 FHIR R4: Observation y Bundle (PI-04) y Patient seudonimizado (PI-05, 05/10). Envío por WebSocket antes que ntfy (−17 % de latencia). Clave de API para sensores y sesión del cuidador con Supabase Auth (05/10). Panel con componentes simulados y alertas atendidas. Oracle Cloud Santiago desplegado el 11/10 (E2.1.Micro, HTTPS; PR-01c 54 ms). Sincronización del modo borde terminada el 08/10 (PR-03b, 5/5).
Realizar pruebas de certificación	**Sprint 5: Certificación técnica.** Pruebas de latencia y carga, validación HL7 FHIR y auditoría de datos.	Postman, Locust, pytest, validador HL7 FHIR	3 semanas (S13–S15)	Andrés Vergara	Facilitador: herramientas de prueba gratuitas. Obstáculo: falsos positivos en los umbrales.	**En curso** (adelantado)	Ejecutadas con resultado satisfactorio: PU-03, PI-02, PI-03 y PI-04. PR-01: 30/30 alertas por WebSocket; meta < 1 s cumplida en AWS EC2 (216 ms) y no en Render (1.577 ms). Validación FHIR con la librería fhir.resources. Agregadas entre el 03 y el 05/10: PR-02 (ntfy, 30/30), PR-03 (sin conexión, 10/10), PU-04 (10/10), PI-03b (RLS en sensores), PI-05 (8/8), PS-01 y PS-01b (3/3 en producción). 06/10: PU-05, PU-06, PI-06 y PI-07. PR-04 (carga con Locust) ejecutada el 20/10 en Oracle Santiago: 20 usuarios, 0 errores, p95 < 200 ms.
Realizar pruebas de certificación;	**Sprint 6: Validación con usuarios y cierre.** Escala SUS, ajustes, informe final	Google Forms, Word, PowerPoint	3 semanas (S16–	Andrés Vergara	Facilitador: red de contactos con cuidadores.	**En curso (adelantado)**	Escala SUS adelantada al Sprint 4: siete cuidadores reales evaluados entre el 15 y el 21/10 (media 92,1; IC 95 %: 83,4 a 100). Sus

Plan de Trabajo							
Competencia o unidades de competencias	Actividades	Recursos	Duración de la actividad	Responsable	Observaciones	Estado de avance	Ajustes
Gestionar proyectos informáticos	y ensayo de defensa.		S18)		Obstáculo: baja participación.		hallazgos generaron las historias HU-20 a HU-23.
Gestionar proyectos informáticos	**Transversal: documentación y seguimiento.** Bitácora semanal, actualización del backlog y commits.	Obsidian, GitHub	18 semanas	Andrés Vergara	Facilitador: registro continuo del avance. Obstáculo: carga académica y laboral.	**Ajustada** (tuvo retraso)	Obsidian no estuvo disponible con la cuenta institucional. Se reemplazó por fichas de sprint y retrospectivas reconstruidas desde los commits (01/10), el tablero Kanban público y actualizado (02/10) y el registro de pruebas. Desde el 02/10: verificación de despliegues después de cada push y actualización semanal del tablero. Desde el 20/10: bitácora semanal en la ficha del sprint en curso y backlog actualizado.

Estados según la plantilla: En curso / Con retraso / No iniciado / Completado / Ajustada. Responsable único: proyecto APT individual.

3. Ajustes a partir del monitoreo

Profundiza en las observaciones de tu plan de trabajo. Analiza las actividades planificadas y señala qué aspectos facilitaron u obstaculizaron la ejecución del plan. Plantea cómo abordaste y/o abordarás los obstáculos. Por último, señala los ajustes que realizaste al plan de trabajo a partir de este análisis.

Factores que han facilitado y/o dificultado el desarrollo de mi plan de trabajo

Facilitadores:

- Mi formación previa como Tecnólogo en Informática Biomédica, que me permitió entender el dominio de salud y sus estándares (HL7 FHIR, SNOMED CT, LOINC).
- La retroalimentación continua del profesor guía, incluso a distancia: el envío de avances cuando no puedo asistir a clases me permite validar decisiones sin esperar a la siguiente sesión.
- Los planes gratuitos de Render, Vercel y Supabase, y los créditos de AWS Academy (USD 50, con un gasto menor a USD 0,10).
- La documentación abierta de FastAPI y React.
- El historial de commits de GitHub, que permitió reconstruir y trazar los principales avances técnicos.

Dificultades y acciones tomadas:

Dificultad	Acción tomada o planificada
Incorporación tardía a la asignatura (19/08, una semana después del inicio).	Creación inmediata del repositorio y priorización del backlog en el flujo crítico (caídas y emergencias).
Carga laboral en turnos rotativos.	Organización del trabajo mediante bloques fijos concentrados en los días libres.
El proveedor de internet bloquea el puerto 5432 de PostgreSQL.	Cambio de Neon a Supabase con Session Pooler IPv4 (puerto 6543).
Límites de los planes gratuitos: Supabase se pausó por inactividad, Render suspende el servidor sin uso y 7 despliegues de Vercel fallaron sin detectarse.	Reactivación de Supabase, corrección del build (commit 9b07078) y nueva práctica: verificar el despliegue después de cada push y abrir el panel cada semana.
Latencia alta por la distancia entre servicios (Render en EE. UU. y Supabase en Brasil).	Comparación medida en un entorno de prueba en AWS EC2 (216 ms). El despliegue en Oracle Cloud Santiago se intentó el 03 y el 05/10, pero la región no tenía capacidad para las máquinas gratuitas; se posterga.
Brechas de seguridad detectadas al probar: la API no tenía autenticación, la tabla de sensores no tenía RLS y en AWS falta https.	Corregidas el 05/10: clave de API para sensores (PS-01), Supabase Auth para el panel (PS-01b) y RLS en sensores (PI-03b). Pendiente: certificado TLS en AWS.
Horas mezcladas en la base y 1.463 alertas atendidas solo en la tabla eventos.	Respaldo y migraciones versionadas el 06/10, medidas antes y después (PI-06 y PI-07).
Al activar la clave de API, la clave guardada no coincidía con la del servidor.	Diagnóstico con una huella SHA-256 parcial publicada en /api/health, sin exponer la clave, y reemplazo por una clave nueva.
Obsidian no disponible con la cuenta institucional.	Registro del trabajo dentro del repositorio GitHub.

Actividades ajustadas o eliminadas

- **HL7 FHIR (adelantada):** se realizó una implementación inicial en el Sprint 3, en vez del Sprint 4 o la etapa post-prototipo, porque el backend ya estaba estable y la interoperabilidad es un requisito clave para integrarse con sistemas de salud.
- **Bitácora en Obsidian (eliminada y reemplazada):** se reemplazó por fichas de sprint, retrospectivas, registro de pruebas y tablero Kanban en GitHub, una herramienta más trazable y accesible para el docente.
- **Despliegue con Docker en AWS EC2 (agregada):** se incorporó como entorno de prueba para comparar el comportamiento del sistema.
- **Despliegue en Oracle Cloud Santiago (agregada y postergada):** a partir de la comparación de latencias, busca acercar el servidor a los usuarios en Chile, con https y sin costo. Se intentó el 03 y el 05/10 sin capacidad gratuita disponible en la región; actualización: desplegado el 11/10 en la forma gratuita E2.1.Micro, con HTTPS (DuckDNS + Caddy); la forma Ampere A1 sigue sin capacidad y queda un reintento automático.
- **Seguridad de la API (agregada):** clave de API para los sensores e inicio de sesión del cuidador con Supabase Auth, guardados como variables de entorno y nunca en el repositorio.
- **Regla de la puerta principal (ajustada):** pasó del simulador al backend y alerta solo de noche o si la puerta queda abierta 10 minutos, para reducir falsas alarmas.
- **Orden de las notificaciones (ajustada):** el WebSocket se envía antes que ntfy, lo que redujo la latencia en 17 %.
- **Rediseño de la base de datos (adelantado):** las etapas 1 y 3 de la ficha técnica 19, planificadas para el Sprint 4, se ejecutaron el 06/10: migraciones versionadas, fechas con zona horaria y alertas como fuente única de la atención.
- **Validación FHIR (ajustada):** se usa la librería fhir.resources, basada en los modelos oficiales FHIR R4, en lugar del validador oficial de HL7, lo que permite automatizar la prueba.

Actividades que no has iniciado o están retrasadas

Actividad	Motivo	Estrategia
Despliegue en Oracle Cloud Santiago	Sin capacidad gratuita en la región (03 y 05/10).	Reintentar al confirmarse la cuenta de pago por uso; si no hay capacidad, se documenta como hallazgo y se mantiene AWS EC2 como comparación.
Sincronización con la nube al volver internet (OE1)	Detectada en PR-03: el modo borde no sube los eventos guardados.	Sprint 5: guardar los eventos como pendientes y reenviarlos a Supabase.
Protección de las consultas de lectura de la API	Las escrituras ya están protegidas; las lecturas dependen de CORS.	Sprint 5, con la misma sesión del cuidador.
Prueba de carga con Locust (PR-04)	Planificada para el Sprint 5.	Se mantiene en S13–S15.
Escala SUS (HU-12)	Planificada para el Sprint 6.	Convocatoria de 5 a 10 cuidadores desde S13.
Bitácora (con retraso)	Herramienta no disponible.	Resuelto: registro reconstruido el 01/10 y actualización semanal desde el 02/10.

Completadas entre el 03 y el 05/10: PR-03 (10/10), PR-02 (30/30), PI-05 (8/8), PS-01 y PS-01b (3/3 en

producción).

Actualización al 20/10: completadas PR-01c (Oracle Santiago, 11/10), HU-03 con PR-03b (08/10), PR-04 (carga, 20/10) y siete sesiones SUS (15 al 21/10; media 92,1). Pendientes: historias HU-20 a HU-23 surgidas del SUS (Sprint 5) y RLS por cuidador (HU-19).

4. Conclusions

The DomoVida prototype already covers the critical flow defined in Phase 1: fall, detection, alert and caregiver response. OE1 and OE2 show a high degree of implementation and validation within the current prototype scope: the edge mode kept working without internet (10/10), the FHIR Patient resource was correctly structured and validated using the fhir.resources library (8/8), and both the sensors and the caregiver are now authenticated (3/3 in production). Automatic synchronization of locally stored edge events when the connection is restored was implemented on October 8 (5 out of 5, PR-03b). OE3 is in progress: 100% of the push notifications reached the phone (30/30), and the SUS evaluation with seven real caregivers reached a mean score of 92.1 (95% CI: 83.4 to 100).

The most important finding of this stage came from measuring instead of assuming. The first latency test did not meet the 1-second goal. Analyzing the cause led to two improvements: reordering the notifications (−17% latency) and comparing infrastructures, which reduced the latency to 216 ms on AWS EC2. Testing also revealed security gaps, such as the API not requiring authentication and the lack of HTTPS on EC2. Instead of remaining hidden, the authentication gap was closed with an API key for the sensors and caregiver login, and a table without RLS found by the Supabase security advisor was protected the same day. Documenting the data model revealed another hidden problem: mixed time zones and 1,463 alerts whose response was recorded in only one table. Both were fixed with versioned database migrations, measured before and after (PI-06 and PI-07).

After the Evaluation 2 feedback, the backend and the database were deployed in the same region (Oracle Cloud Santiago, 54 ms), the load test passed (20 concurrent users, 0 errors, p95 below 200 ms), the repository was cleaned up, and the agile documentation is now updated weekly. The SUS evaluation was completed with seven caregivers (mean 92.1). The next step is to implement the improvements it revealed: a traffic-light status view, the connection status of each sensor, better text contrast and a log of the hired caregiver's tasks.

5. Reflection

This phase taught me that a working prototype is not the same as a validated one. Some of my most valuable results came from tests that failed, such as the latency goal on Render, the consent process tested over plain HTTP, or the free database that paused itself. Each failure forced me to understand my own system more deeply.

I also learned the value of documenting while working. I did not keep a weekly log, and rebuilding three sprints from commits took a full day. That is why I now update the Kanban board and the test record as part of each task, not at the end, and after the Evaluation 2 feedback I added a weekly log to the current sprint.

Finally, this project confirmed my motivation. Bringing together my background in biomedical informatics and software engineering to support older adults and their caregivers gives meaning to the technical work. It is also the foundation for continuing DomoVida beyond the course.

6. Declaración de uso de inteligencia artificial

En el desarrollo de este proyecto utilicé Claude (Anthropic, 2026) como asistente para proponer y revisar código, explicar conceptos técnicos, redactar borradores de documentación y diseñar diagramas. Todas las decisiones de diseño, alcance y prioridad, la ejecución de los cambios, las pruebas en producción y la validación de los resultados fueron realizadas por mí. Revisé y comprendí cada propuesta antes de incorporarla al proyecto, y soy responsable de su contenido.

7. Referencias

Anthropic. (2026). Claude [Modelo de lenguaje de gran tamaño]. https://claude.ai

Bangor, A., Kortum, P., y Miller, J. (2009). Determining what individual SUS scores mean: Adding an adjective rating scale. Journal of Usability Studies, 4(3), 114–123.

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester y I. L. McClelland (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis.

Docker Inc. (2024). Docker documentation. https://docs.docker.com

Fette, I., y Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6455

Heckel, P. C. (2024). ntfy documentation. https://docs.ntfy.sh

Hipp, D. R. (2024). SQLite documentation. https://www.sqlite.org/docs.html

HL7 International. (2019). FHIR R4: Fast Healthcare Interoperability Resources (versión 4.0.1). https://hl7.org/fhir/R4/

Locust. (2025). Locust documentation (versión 2). https://docs.locust.io

Meta Platforms. (2024). React documentation. https://react.dev

National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS) (FIPS PUB 180-4). https://doi.org/10.6028/NIST.FIPS.180-4

Oracle. (2026). Oracle Cloud Infrastructure documentation. https://docs.oracle.com/en-us/iaas/Content/home.htm

Ramírez, S. (2023). FastAPI: documentación oficial. https://fastapi.tiangolo.com

Regenstrief Institute. (2024). LOINC: Logical Observation Identifiers Names and Codes. https://loinc.org

República de Chile. (2024). Ley N° 21.719: Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. Biblioteca del Congreso Nacional. https://www.bcn.cl/leychile/navegar?idNorma=1209272

Sauro, J. (2011). A practical guide to the System Usability Scale: Background, benchmarks & best practices. Measuring Usability LLC.

Schwaber, K., y Sutherland, J. (2020). La guía de Scrum: La guía definitiva de Scrum — Las reglas del juego. Scrum.org. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf

SNOMED International. (2024). SNOMED CT. https://www.snomed.org

Supabase Inc. (2024). Supabase documentation. https://supabase.com/docs